

Malicious URL/Phishing Detector

I. Problem Statement

Phishing is a dangerous scam that is used to steal people's information, including passwords, credit cards, and personal data. This is also extended to companies, where hackers will try to steal corporate data, which could result in millions of people's data being leaked. With the continuous growth in Phishing attacks across different use cases (email, suspicious links, etc.) it is becoming important to detect phishing quickly in order to scale with the rate it is being used. We seek to determine if Bloom Filtering is an efficient and scalable way to deal with the problem.

II. Dataset Description

Using the "[Database of Benign and Malicious Webpages](#)" on Kaggle, we put the dataset in a folder labeled "data" in the repository. The CSV file has 12 different labels: **index** (The label is actually blank in the CSV, but that is what the label represents), **url**, **url_len** (URL length), **ip_add** (IP address), **geo_loc** (Geolocation), **tld** (Top Level Domain), **who_is** (Whether there are accessible records regarding the person or entity that registered the domain), **https** (Whether the URL has SSL/TLS encryption), **js_len** (Total amount of JavaScript code), **js_obf_len** (The amount of code where functionality is hidden), **content**, **label**. The label is set to either "good", meaning a benign webpage, or "bad", meaning a malicious webpage. Of these labels, we will only be using "label" and "url", as they are the only information necessary to populate and query a Bloom Filter.

III. Data Processing Steps

We decided that we were going to use stream.jar from Homework 3 to simulate streaming, so we converted the CSV file to a JSON file to mimic the yelp.json file in Homework 3. Furthermore, to make it

easier to populate the Bloom filter, we made a file that only contains the url and label for malicious links, omitting the benign ones so the Bloom filter will not have excess to deal with data when it is getting set up before testing.

IV. Streaming Pipeline

As mentioned above, we are using stream.jar from Homework 3 for stream simulation. We implemented a function called start_streaming() that will start Spark's socketTextStream, call process_rdd() on every line in the JSON file, and then wait for termination. The process_rdd() function will collect all of the urls in each batch and query the Bloom Filter to see if they are malicious or not.

V. Overall System Architecture

The system is split into two versions of the Bloom Filter: one that is offline and one that is online. This decision was made in order to make the application more responsive, as it entirely eliminates any waiting the user would have to do for the Bloom Filter to populate. The offline version of the Bloom Filter will populate the bit array with all of the malicious URLs in the dataset, and save this bit array to a .bin file to later be referenced by the online version. The online version is what is actually used when the user runs the application, where the input URLs are checked against the saved bit array to see if they have been seen before or not.

This also means that we have separate files for each function: populating_bloom_filter() is used to populate the Bloom Filter, and phishing_detector.py is used to run the Bloom Filter on streaming data.

VI. Bloom Filter Implementation

We implemented a Bloom Filter class, which has the methods **add()**, **check()**, **_hashes()**, **get_fill_ratio()**, **get_num_bits_set()**, **save_bit_array()**, and **load_bit_array()**. Since we need to run the Bloom filter twice, once offline to set up the bit array and once online to test functionality, this makes sure the code is reusable and clean.

Since we have two implemented hash functions, MurmurHash3 and Jenkins, the hash function used can be specified in the init of the Bloom Filter itself. The correct hash function will then be called by `_hashes()`, which will return a list of all of the indexes in the bit array that should be set to 1.

`add()` is a function to add the given URL into the bit array, which will only be called during the setup when given a list of entirely malicious URLs. It is given a URL and will call `_hashes()` to get the indices that it will set to 1. `check()` is only called during the online streaming test part of the code, which will use the hash functions to check whether or not a URL is in the array and return a boolean.

`save_bit_array()` and `load_bit_array()` are necessary functions because we need to maintain a global bit array for both online and offline. These functions allow us to save the bit array created offline to a file, and then load that file when we are testing online to prevent delays in checking a URL.

VII. Hashing Algorithms

A. MurmurHash3

MurmurHash3 is a non cryptographic hashing function that's often used for Bloom Filters. It works by converting the input into bytes, splitting it into 4-byte chunks, and within each chunk it packs four bytes into a single integer in little-endian order. Then for each integer it makes it will multiply it by a constant mixer value, rotate the bits in a circular left shift, and then multiply this value by another constant mixer value. After this, it merges the value into the hash using an XOR function, and rotates it again. Finally it will randomize the state with a linear transformation. After all chunks are processed, it will hash any remaining bytes as a partial chunk and then hash it in the same way. The last part of this function is to mix in the length so that different-length inputs will not easily collide and then do a series of XORs, shifts, and multiplications to ensure that every input bit will affect every output bit. Throughout this entire process, it makes sure to AND any values with 0xFFFFFFFF to make sure that the value will stay within 32 bits.

B. Jenkins Hash

Jenkins Hash is another non cryptographic hashing function often used for Bloom Filters. It works by starting the hash with a seed and then looping through the input string one character at a time. For each character, it will convert it to its ASCII/Unicode number and add it into the hash. It then shifts all bits in the hash to the left by 10 and adds that back into the hash, in order to allow early bits to have an influence. It then does an XOR with all the bits shifted to the right by 6, which will mix the high bits into the low bits. After it does this for every character, it does its final mixing, where it shifts all bits to the left by 3 and adds that into the hash, shifts all bits to the right by 11 and XORs that with the hash, and then finally shifts all bits to the left by 15 and adds that into the hash.

VIII. Experimental Setup

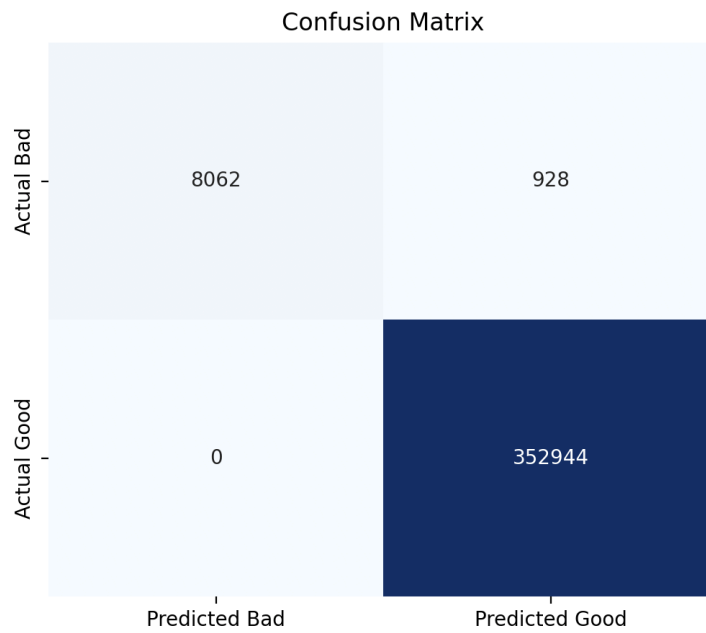
For this experiment, we are comparing the performance of two hash functions: MurmurHash3 and Jenkins Hash. The data is set up by files like `csv_to_json.py` and `filter_malicious_webpages.py`, which will convert the data into a JSON and create a separate JSON file of entirely malicious links, respectively. In order to set up the bit arrays, `populating_bloom_filter.py` will be run on both the MurmurHash3 and Jenkins hash. Finally, all of the data in the dataset will be checked by the Bloom Filter to classify every link. False positives, true positives, false negatives, and true negatives will all be kept track of and printed out for the user to see. This can either be done online (through `phishing_detector.py`) or offline (through `checking_accuracy.py`). m (number of URLs inserted into array), n (number of bits in array), and k (number of hash functions) are all the same for each implemented hash, ensuring the only thing being tested in this scenario is how accurate the hash function itself is.

IX. Evaluation Metrics

The two evaluation metrics that we used are **False Positive Rate (FPR)** and **execution time**. Execution time is only run on the offline version of the Bloom Filter, as the difference between

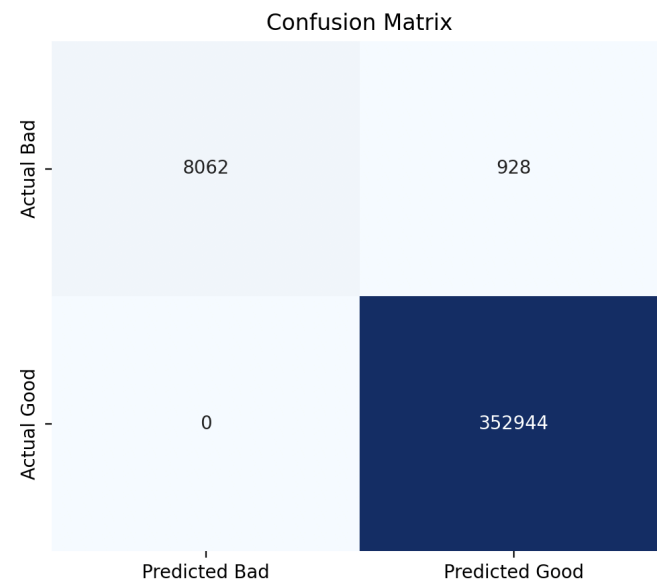
classifying the number of links in each batch (10) would be negligible, and streaming takes quite a while to run due to the speed at which links come in and the number of links that need to be classified.

A. *MurmurHash3*



Execution time (offline): 18.3522 seconds

B. *Jenkins Hash*



Execution Time (offline): 18.7168 seconds

C. Analysis and Interpretation

Both hashing algorithms were able to accurately classify all 8062 malicious links in the dataset as malicious, meaning that this filter will be able to consistently catch every single known malicious link it is provided with. Furthermore, since both algorithms produced zero false negatives, we can see that the filters are working as expected, since Bloom Filters inherently do not produce false negatives. They also had very similar execution times, with only a 0.4 second difference when classifying 361,933 URLs.

The main thing to take note of is that both hashing algorithms have the same FPR— falsely classifying 928 URLs as malicious, meaning that the FPR is 0.0026. This is interesting to consider because the expected behavior is that the FPR would be different with two entirely different hashing algorithms. However, investigating this shows that it is merely a coincidence, likely because both Bloom Filters have the same exact parameters for m , n , and k . The lists of false positives are nearly entirely distinct from each other, with only 9 shared links between them. This implies that both filters are operating independently of each other, as they have different collisions that result in different false positives.

This FPR is also below what the calculated FPR is, which is 0.0181. This is assumed to have happened because the hash functions had a uniform distribution of URLs, resulting in fewer collisions than expected.

X. Challenges

The main challenge with this project was the hash functions. We had no prior experience with implementing hash functions, and very little knowledge of what hash functions are actually used for Bloom Filters. This means that we had to do a lot of research about which hash functions are commonly used, what they do, and how they work in order for us to implement them. It was also quite difficult to understand exactly what is happening in every part of the hash function, because most of it is just different types of bit shifting, and the specifics of why each operation was picked in each spot required further research.

We also used new libraries in this project, most notably PySimpleGUI. While it was simple enough to use after getting used to it, going through the documentation and figuring out what we needed in order to accomplish our idea of what the interface would look like took a lot of time.

XI. Improvements

Since Bloom Filters only work on known malicious links, any new malicious links or ones that have flown under the radar will not be correctly identified by the Bloom Filter despite its accuracy. Knowing this, it would be interesting to add a way to identify new malicious links through a machine learning model trained on all of the unused labels in the dataset we used, specifically things like **https**, **js_obf_len**, **tld**, and **content**. These labels are chosen specifically because they provide a lot of information that can be used to determine if a webpage is malicious or not: **https** being true prevents being scammed by outside sources; a high **js_obf_len** value means that a lot of JavaScript code is being hidden, which could signal malicious intent; known safe websites will have a known **tld** so they can be ruled out; and **content** will provide everything on the page, including JavaScript, which can be compared to the content on known malicious sites to see if they are similar. This change would make the Bloom Filter less static, allowing it to add to its database of known malicious links over time instead of being stuck with the same set forever.

Other small improvements are implementing more hash functions, improving the GUI, and extending the functionality to other parts of the internet. Implementing more hash functions means that there is more data to compare in order to determine which hash function is the best choice for a Bloom Filter. GUI can always be improved, adding more visual interest and making the user experience better. Finally, extending functionality to other parts of the internet, like the Google search page, could be done through something like a Chrome extension. This would allow the user to feel more secure while browsing the entire internet, not just in their mail inbox.

XII. Key Findings

Through this experiment, we found that the MurmurHash3 and Jenkins Hash have comparable performances on the same data, as their execution time and accuracy were very similar to each other. We also found that the Bloom Filter is a suitable data structure for preventing Phishing attacks, as it can handle a lot of data very efficiently and accurately, and was only held back when simulating streaming because of the speed at which URLs came in. This means that if URLs came in at higher speeds, the Bloom Filter would be able to keep up with demand and continue to classify them properly.

The implemented GUI also shows an example of how they can be used in the real world, as it filters messages that contain malicious URLs so the user will not see them unless they choose to.

XIII. Individual Contributions

A. Taylor

1. Data Pre-Processing
2. Bloom Filter Implementation
3. GUI
4. Final Report

B. Jeron

1. MurmurHash3 Implementation

C. Tati

1. Jenkins Hash Implementation

D. Group

1. Streaming Pipeline Implementation
2. Presentations
3. Reports 1-3

XIV. References

“Non-Cryptographic Hash Function.” *Wikipedia.Org*, Wikimedia Foundation, Inc., 9 Apr. 2009,
<https://en.wikipedia.org/wiki/MurmurHash>. Accessed 4 May 2026.

“MurmurHash3.” *GitHub*, Aug. 2016,
<https://github.com/aappleby/smhasher/wiki/MurmurHash3#bulk-speed-test-hashing-an-8-byte-aligned-256k-block>. Accessed 4 May 2026.

“Collection of Hash Functions.” *Wikipedia.Org*, Wikimedia Foundation, Inc., 3 Apr. 2009,
https://en.wikipedia.org/wiki/Jenkins_hash_function. Accessed 4 May 2026.

XV. GitHub Link

<https://github.com/taylor-musso/phishing-detector/tree/40bb291491dd9a41bbaf0778894d892c7f4c37a2>